IF3170 Inteligensi Artifisial

LAPORAN TUGAS BESAR 1

Disusun oleh:

Nathan E C Marpaung — 13524062

Kurt Mikhael Purba — 13524065

Aurelia Jennifer Gunawan — 13524089

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

Daftar Isi

1 Deskripsi Persoalan **1**

2 Dasar Teori **3**
2.1 Algoritma *Hill Climbing* . . . 3
2.1.1 *Steepest Ascent Hill Climbing* . . . 3
2.1.2 *Hill Climbing with Sideways Move* . . . 4
2.1.3 *Random Restart Hill Climbing* . . . 5
2.1.4 *Stochastic Hill Climbing* . . . 7
2.2 Algoritma *Simulated Annealing* . . . 8
2.3 Algoritma *Genetic Algorithm* . . . 10

3 Implementasi **12**
3.1 Arsitektur dan Struktur Program . . . 12
3.2 Representasi State, Fungsi Objektif, dan Constraints . . . 12
3.3 Representasi Neighbor dan State Awal . . . 12
3.4 Implementasi Kelas-kelas Pendukung . . . 12
3.5 Implementasi Algoritma Hill-climbing . . . 12
3.6 Implementasi Algoritma Simulated Annealing . . . 12
3.7 Implementasi Algoritma Genetic Algorithm . . . 12

4 Hasil Eksperimen dan Analisis **13**
4.1 Eksperimen Algoritma *Hill-climbing* . . . 13
4.1.1 *Steepest Ascent Hill-climbing* . . . 13
4.1.2 *Hill-climbing with Sideways Move* . . . 13
4.1.3 *Random Restart Hill-climbing* . . . 13
4.1.4 *Stochastic Hill-climbing* . . . 13
4.2 Eksperimen Algoritma *Simulated Annealing* . . . 13
4.3 Eksperimen Algoritma *Genetic Algorithm* . . . 13
4.4 Analisis Hasil Eksperimen . . . 13

5 Kesimpulan dan Saran **14**
5.1 Kesimpulan . . . 14
5.2 Saran . . . 14

Referensi **15**

Lampiran **16**
Pembagian Tugas 16
Form Pengunaan AI 17

Bab 1

Deskripsi Persoalan

Pemuatan barang ke dalam kendaraan angkut merupakan salah satu permasalahan umum dalam industri logistik yang melibatkan berbagai variabel, seperti dimensi paket, berat, nilai barang, kerapuhan, keterbatasan ruang, dan kapasitas kendaraan. Penentuan paket yang dimuat dan penempatannya dalam peti kemas truk memerlukan pengaturan yang efisien agar tidak terjadi pelanggaran, seperti paket yang melayang atau menembus dinding peti kemas, barang pecah-belah yang tertindih paket lain, atau total berat muatan yang melebihi kapasitas truk. Apabila dilakukan secara manual, proses ini sangat tidak efektif karena ruang solusi yang meningkat secara eksponensial seiring bertambahnya jumlah paket, dengan kombinasi pilihan paket, posisi koordinat, dan enam kemungkinan orientasi untuk setiap paket.

Tugas besar 1 mata kuliah IF3170 Inteligensi Artifisial bertujuan agar peserta kuliah memahami dan mampu mengimplementasikan serta mengevaluasi berbagai algoritma *local search* untuk menyelesaikan permasalahan pengisian paket pada truk peti kemas Mejiro Express. Persoalan ini merupakan variasi dari *3D bin packing* dan *knapsack problem*, dengan tujuan memaksimalkan total nilai paket yang berada dalam peti kemas. Tidak seluruh paket harus dimuat, sehingga paket yang tidak terpilih dapat ditempatkan di luar truk. Dalam konteks inteligensi artifisial, proses ini dapat dimodelkan sebagai sebuah *problem-solving agent* yang bertindak secara rasional dengan mempertimbangkan keterbatasan ruang dan kapasitas berat. Agen diharapkan dapat memilik aksi terbaik berdasarkan *state* saat ini untuk mencapai kondisi optimal.

Untuk mencapai tujuan tersebut, digunakan pendekatan *local search*. *Local search* cocok untuk masalah yang tidak membutuhkan pencarian jalur menuju solusi dan hanya berorientasi pada pencarian *final state* terbaik. Pencarian dimulai dari *initial state* yang dibangkitkan secara acak, kemudian dieksplorasi terhadap *state* tetangga untuk memperoleh nilai *objective function* yang lebih baik. Perpindahan antar-*state* dibatasi pada tiga jenis langkah, yaitu menukar dua paket, memindahkan sebuah paket ke koordinat yang berbeda, dan memutar paket 90 derajat terhadap sumbu x, y, atau z. Setiap *state* harus memenuhi batasan yang ditetapkan, yaitu paket tidak boleh melayang atau menembus dinding peti kemas, barang pecah-belah tidak boleh menjadi penyangga paket lain, dan total berat muatan tidak boleh melebihi kapasitas truk.

Dalam tugas besar 1, dilakukan implementasi beberapa varian algoritma *local search*, yaitu *Hill-climbing, Simulated Annealing*, dan *Genetic Algorithm*. Setiap algoritma memiliki karakteristik dan strategi pencarian yang berbeda. *Hill-climbing* berfokus pad aperbaikan bertahap menuju solusi terbaik, *Simulated-Annealing* mengombinasikan *Hill-climbing* dengan probabilitas berpindah ke solusi yang lebih buruk untuk menghindari *local optimum*, serta *Genetic Algorithm* yang mempertahankan sekumpulan solusi dan memperbaruinya melalui proses seleksi, *crossover*, dan mutasi. Hasil dari setiap algoritma kemudian dianalisis berdasarlan nilai *objective function* akhir, durasi pencarian, dan konsistensi hasil dari beberapa kali percobaan.

Bab 2

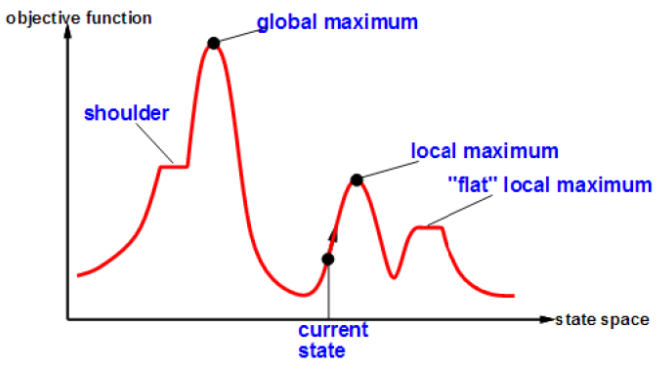
Dasar Teori

Local search merupakan pendekatan dalam kecerdasan buatan yang mencari solusi dengan berpindah dari satu *state* ke *state* tetangganya. *Classical search*, seperti BFS dan DFS, mementingkan jalur menuju solusi. Sedangkan *local search* hanya mempertahankan satu *state* aktif sehingga cocok untuk masalah yang hasil akhirnya lebih penting daripada urutan langkahnya. Setiap *state* merepresentasikan konfigurasi solusi yang lengkap. Perpindahan dilakukan dengan menghasilkan *successor* atau *neighbor* dari *state* saat ini, lalu kualitasnya dinilai menggunakan fungsi objektif. Algoritma akan memaksimalkan fungsi objektif atau meminimalkan fungsi biaya. Tujuan utama dari *local search* adalah untuk menemukan *global optimum*. Namun, pencarian dapat terhenti atau terjebak pada *local optimum*, yaitu *state* yang lebih baik daripada semua tetangganya tetapi bukan yang terbaik secara keseluruhan. Untuk mengatasi permasalahan ini, sebagian algoritma menambahkan mekanisme seperti langkah acak, penerimaan solusi yang sementara lebih buruk, atau pengembangan banyak kandidat sekaligus.

2.1 Algoritma *Hill Climbing*

2.1.1 *Steepest Ascent Hill Climbing*

Steepest ascent hill-climbing adalah varian dasar algoritma pencarian *hill-climbing*, yaitu algoritma pencarian lokal yang bergerak secara terus-menerus ke arah nilai yang lebih baik, baik berupa kenaikan nilai objektif maupun penurunan nilai biaya (*cost*).



Gambar 2.1: Grafik Fungsi Objektif terhadap State Space pada Steepest Ascent HC

Algoritma diawali dari sebuah *initial state* yang dibangkitkan secara acak. Pada setiap iterasi, algoritma membangkitkan seluruh *successor* dari *state* saat ini, lalu memilih *neighbor* yang merupakan *highest-valued successor*, yaitu *successor* dengan nilai tertinggi. Jika terdapat lebih dari satu *successor* terbaik, pemilihan dilakukan secara acak di antara *successor* tersebut. Berikut adalah pseudocode algoritma *steepest ascent hill-climbing*:

Algorithm 1: Hill-Climbing (Steepest Ascent)

Input: *problem*
Output: state yang merupakan local maximum

```
current ← MAKE-NODE(problem.INITIAL-STATE)
while true do
    neighbor ← a highest-valued successor of current
    if neighbor.VALUE ≤ current.VALUE then
        return current.STATE
    current ← neighbor
```

Alur kerja algoritma *steepest ascent HC* terdiri dari tiga bagian. Pertama, inisialisasi dengan *state* acak. Kedua, *loop* yang terus berpindah ke *neighbor* terbaik selama nilainya lebih tinggi daripada *state* saat ini. Ketiga, terminasi ketika tidak ada *neighbor* yang memiliki nilai lebih tinggi dari *state* saat ini. Kondisi ini mencakup puncak maupun area datar, karena kondisi berhenti menggunakan operator $\leq$.

2.1.2 *Hill Climbing with Sideways Move*

Hill-climbing with sideways move adalah salah satu varian dari *steepest ascent hill-climbing* yang dirancang untuk mengatasi kegagalan algoritma dasar pada kondisi *shoulder*. Pada *steepest ascent HC*, pencarian berhenti ketika tidak ada *neighbor* yang bernilai lebih tinggi dari *state* saat ini. Kondisi berhenti ini juga terpicu pada area datar, padahal pada area tersebut, seperti *shoulder*, masih terdapat kemungkinan untuk mencapai nilai yang lebih baik jika pencarian dilanjutkan.

Algoritma HC dengan *sideways move* ini mengizinkan algoritma berpindah ke *neighbor* yang nilainya sama dengan *state* saat ini, yang disebut *sideways move*. Perubahan ini diwujudkan dengan mengganti kondisi terminasi dari $neighbor.Value \leq current.Value$ menjadi $neighbor.Value < current.Value$. Dengan perubahan tersebut, algoritma hanya berhenti apabila seluruh *neighbor* memiliki nilai yang benar-benar lebih rendah, sehingga perpindahan ke *neighbor* bernilai sama tetap diperbolehkan.

Perpindahan sideways tanpa batas dapat menyebabkan *infinite loop*, yaitu ketika algoritma terus berpindah antar-*state* bernilai sama pada *flat local maximum* yang bukan *shoulder*. Maka, jumlah *sideways move* yang dilakukan secara berurutan perlu dibatasi dengan suatu nilai batas. Apabila batas tersebut tercapai, algoritma dihentikan dan mengembalikan *state* saat ini. Nilai limit menjadi parameter yang memengaruhi keseimbangan antara peluang keberhasilan dan waktu komputasi. Berikut adalah pseudocode algoritma *hill-climbing with sideways move*:

Algorithm 2: Hill-Climbing dengan Sideways Move

Input: *problem*, *maxSideways*

Output: state yang merupakan local maximum

```
current ← MAKE-NODE(problem.INITIAL-STATE)
sideways ← 0
while true do
    neighbor ← a highest-valued successor of current
    if neighbor.VALUE < current.VALUE then
        return current.STATE
    if neighbor.VALUE = current.VALUE then
        sideways ← sideways + 1
        if sideways > maxSideways then
            return current.STATE
    else
        sideways ← 0
    current ← neighbor
```

2.1.3 *Random Restart Hill Climbing*

Random restart hill-climbing adalah salah satu varian *hill-climbing* yang mengatasi masalah terjebak pada optimum lokal dengan cara mengulang pencarian dari *state* awal yang berbeda. Algoritma ini menjalankan serangkaian *hill-climbing*, dan setiap pencarian dimulai dari *initial state* yang dibangkitkan secara acak. Proses diulang hingga ditemukan *state* yang memenuhi tujuan, yaitu *state* dengan nilai *global maximum*. Jika pencarian dimulai dari area yang mengarah ke *local maximum*, algoritma akan berhenti di sana dan tidak dapat keluar. Dengan memulai ulang dari *state* acak yang berbeda, setiap percobaan memiliki peluang tersendiri untuk berada pada area yang mengarah ke *global maximum*. Semakin banyak percobaan dilakukan, semakin besar peluang setidaknya satu di antaranya berhasil. Setiap percobaan merupakan

hill-climbing biasa dan bersifat independen terhadap percobaan sebelumnya.

Kinerja algoritma ini dapat dianalisis dengan probabilitas. Misalkan p adalah probabilitas sebuah percobaan hill-climbing berhasil mencapai solusi, s adalah rata-rata jumlah langkah pada percobaan yang berhasil, dan f adalah rata-rata jumlah langkah pada percobaan yang gagal. Maka jumlah *restart* yang diharapkan hingga ditemukan solusi adalah

$$E[\text{percobaan}] = \frac{1}{p} \tag{2.1}$$

dan jumlah langkah yang diharapkan adalah

$$E[\text{langkah}] = s + \frac{f(1-p)}{p} \tag{2.2}$$

Persamaan kedua diperoleh dari dua komponen. Rata-rata terdapat $\frac{1-p}{p}$ percobaan gagal sebelum percobaan pertama yang berhasil, dan tiap percobaan gagal membutuhkan f langkah. Percobaan terakhir yang berhasil menambah s langkah. Dari persamaan tersebut terlihat bahwa nilai p yang kecil membuat jumlah restart membesar, tetapi total langkah tetap dapat rendah selama setiap percobaan berlangsung singkat. Hill-climbing yang cepat berhenti sehingga cocok digunakan secara berulang.

Algoritma ini juga bersifat *complete* secara probabilistik. Selama $p > 0$, solusi pada akhirnya akan ditemukan dengan probabilitas mendekati 1 jika jumlah restart tidak dibatasi. Namun, waktu komputasi tidak dapat dipastikan karena bergantung pada keberuntungan pembangkitan state awal. Berikut adalah pseudocode algoritma *random restart hill climbing*

Algorithm 3: Random Restart Hill-Climbing

Input: *problem*

Output: state dengan nilai global maximum

```
while true do
    current ← MAKE-NODE(RANDOM-STATE(problem))
    while true do
        neighbor ← a highest-valued successor of current
        if neighbor.VALUE ≤ current.VALUE then
            break
        current ← neighbor
    if current.VALUE = problem.GLOBAL-MAXIMUM then
        return current.STATE
```

2.1.4 *Stochastic Hill Climbing*

Stochastic hill-climbing adalah salah satu varian *hill-climbing* yang tidak mengevaluasi seluruh *successor* dari *state* saat ini. Pada *steepest ascent HC*, algoritma membangkitkan seluruh *successor* lalu memilih yang bernilai tertinggi. Pada varian ini, algoritma hanya membangkitkan satu *successor* secara acak, kemudian memutuskan apakah perpindahan dilakukan berdasarkan perbandingan nilainya dengan *state* saat ini. Algoritma akan berpindah ke *neighbor* acak tersebut hanya jika nilainya lebih baik daripada *state* saat ini. Jika *neighbor* yang terpilih lebih buruk atau sama, *neighbor* tersebut diabaikan dan algoritma mencoba membangkitkan *successor* acak yang lain.

Algoritma tidak dapat menentukan bahwa ia telah mencapai puncak hanya dari satu kali percobaan yang gagal, karena tidak ada jaminan bahwa *successor* acak yang dipilih lebih baik. Oleh karena itu, kondisi terminasi adalah tercapainya jumlah iterasi maksimum (n_{max}). Algoritma berhenti setelah n_{max} iterasi dan mengembalikan *state* terakhir yang dicapai. Dibandingkan dengan *steepest ascent*, *stochastic hill-climbing* memerlukan biaya per iterasi yang lebih rendah karena hanya satu *successor* yang dievaluasi, tetapi secara keseluruhan membutuhkan lebih banyak langkah untuk mencapai hasil yang sebanding sehingga berjalan lebih lambat. Algoritma ini tetap dapat terjebak pada *local maximum*, karena hanya menerima perpindahan yang memperbaiki nilai. Berikut adalah pseudocode algoritma *stochastic hill-climbing*:

Algorithm 4: Stochastic Hill-Climbing

Input: *problem*, n_{max}

Output: state hasil pencarian

```
current ← MAKE-NODE(problem.INITIAL-STATE)
for i ← 1 to n_max do
    neighbor ← a random successor of current
    if neighbor.VALUE > current.VALUE then
        current ← neighbor
return current.STATE
```

2.2 Algoritma *Simulated Annealing*

Simulated annealing adalah algoritma *local search* berupa versi *stochastic hill-climbing* yang mengizinkan sebagian langkah buruk agar dapat keluar dari *local maximum*. Frekuensi langkah buruk ini dikurangi secara bertahap selama pencarian berlangsung. Nama algoritma ini berasal dari proses *annealing* pada metalurgi, yaitu pemanasan logam lalu pendinginan perlahan. Analoginya, parameter T berperan sebagai "temperatur" yang bernilai tinggi di awal dan menurun mengikuti jadwal pendinginan, yaitu fungsi yang memetakan waktu t ke nilai T.

Pada setiap iterasi, algoritma memilih satu successor acak dari *state* saat ini, lalu menghitung selisih nilainya, yaitu $\Delta E = next.\text{VALUE} - current.\text{VALUE}$. Jika $\Delta E > 0$, *neighbor* lebih baik dan algoritma selalu berpindah. Jika $\Delta E < 0$, *neighbor* lebih buruk, tetapi algoritma tetap dapat berpindah dengan probabilitas $e^{\Delta E/T}$. Probabilitas ini makin kecil ketika langkah semakin buruk dan ketika T semakin rendah. Akibatnya, pada T tinggi algoritma berperilaku seperti *random walk*, sedangkan pada T mendekati 0 algoritma berperilaku seperti *stochastic hill-climbing*. Algoritma berhenti ketika $T = 0$ dan mengembalikan *state* saat ini.

$$\Delta E = next.\text{VALUE} - current.\text{VALUE} \tag{2.3}$$

Keputusan perpindahan bergantung pada tanda ΔE. Jika $\Delta E > 0$, *neighbor* lebih baik dan algoritma selalu berpindah. Jika $\Delta E < 0$, *neighbor* lebih buruk, tetapi algoritma tetap dapat berpindah dengan probabilitas

$$P = e^{\Delta E/T} \tag{2.4}$$

Karena $\Delta E < 0$, nilai P selalu berada di antara 0 dan 1. Probabilitas ini menurun ketika langkah semakin buruk (nilai $|\Delta E|$ makin besar) dan ketika temperatur semakin rendah. Pada temperatur tinggi, P mendekati 1 sehingga hampir semua langkah buruk diterima. Pada temperatur mendekati 0, P mendekati 0 sehingga langkah buruk hampir tidak pernah diterima.

Untuk menentukan apakah langkah buruk diterima, nilai P dibandingkan dengan suatu nilai pembanding. Terdapat dua pendekatan, yaitu membandingkan dengan nilai statis (misalnya langkah diterima jika $P > 0{,}5$), atau membandingkan dengan bilangan acak pada rentang $[0, 1]$ (langkah diterima jika $P > \text{random}(0, 1)$). Pendekatan kedua lebih sesuai dengan sifat stokastik algoritma karena langkah buruk yang kecil tetap memiliki peluang diterima secara proporsional dengan nilai P.

Algoritma berhenti ketika $T = 0$ dan mengembalikan *state* saat ini. Secara teoritis, apabila T diturunkan cukup lambat, *simulated annealing* akan menemukan global optimum dengan probabilitas mendekati 1. Kelemahannya, penurunan temperatur yang lambat membuat waktu pencarian lebih lama, sehingga pemilihan jadwal pendinginan menjadi parameter penting. Berikut adalah pseudocode algoritma *simulated annealing*:

Algorithm 5: Simulated Annealing

Input: *problem*, *schedule*
Output: state solusi

```
current ← MAKE-NODE(problem.INITIAL-STATE)
for t ← 1 to ∞ do
    T ← schedule(t)
    if T = 0 then
        return current.STATE
    next ← a randomly selected successor of current
    ΔE ← next.VALUE − current.VALUE
    if ΔE > 0 then
        current ← next
    else
        current ← next only with probability e^(ΔE/T)
```

2.3 Algoritma *Genetic Algorithm*

Genetic algorithm (GA) adalah algoritma *local search* yang mempertahankan sekumpulan *state* (populasi). Genetic algorithm membangkitkan *successor* melalui penggabungan dua *state* induk (*parent*). Setiap *state*, yang disebut individu, direpresentasikan sebagai *string* pada alfabet berhingga, umumnya *string* bernilai 0 dan 1. Setiap karakter pada *string* merepresentasikan satu variabel permasalahan. Kualitas setiap individu dinilai oleh fungsi evaluasi yang disebut fungsi kebugaran (*fitness function*), dengan nilai yang lebih tinggi menunjukkan *state* yang lebih baik. Fungsi *fitness* harus dirancang sesuai permasalahan, dan nilai maksimumnya menjadi global maximum.

Algoritma diawali dengan populasi awal berisi k individu yang dibangkitkan secara acak. Generasi berikutnya dibentuk melalui tiga operasi berikut.

1. **Selection**

   Dua individu induk dipilih secara acak dengan probabilitas proporsional terhadap nilai *fitness*. Probabilitas terpilihnya individu i adalah

   $$P(i) = \frac{F(i)}{\sum_{j=1}^{k} F(j)} \tag{2.5}$$

   Individu dengan *fitness* lebih tinggi lebih berpeluang terpilih, tetapi individu dengan *fitness* rendah tetap memiliki peluang. Implementasinya dapat berupa *roulette wheel*, yaitu membangkitkan bilangan acak pada rentang $[0, 100]$ lalu mencocokkannya dengan interval kumulatif probabilitas setiap individu.

2. **Crossover**

   Sebuah titik potong (*crossover point*) dipilih secara acak pada string. Anak dibentuk dengan menggabungkan bagian awal *string* induk pertama dan bagian akhir *string* induk kedua. Jika dilakukan pada kedua arah, dihasilkan dua anak.

3. **Mutation**

   Dengan probabilitas kecil, satu karakter pada *string* anak diubah secara acak pada titik mutasi (*mutation point*). Mutasi menjaga keragaman populasi dan memungkinkan munculnya karakteristik yang tidak dimiliki kedua induk.

Proses selection, crossover, dan mutation diulang hingga terbentuk populasi baru

berukuran sama dengan populasi sebelumnya, kemudian populasi lama digantikan. Iterasi berlanjut hingga ada individu yang cukup fit atau waktu yang ditentukan habis. Algoritma mengembalikan individu terbaik pada populasi menurut fungsi *fitness*. Berikut adalah pseudocode algoritma *genetic algorithm*:

Algorithm 6: Genetic Algorithm

Input: *population*, FITNESS-FN
Output: individu terbaik

```
repeat
    new_population ← ∅
    for i ← 1 to SIZE(population) do
        x ← RANDOM-SELECTION(population, FITNESS-FN)
        y ← RANDOM-SELECTION(population, FITNESS-FN)
        child ← REPRODUCE(x, y)
        if probabilitas acak kecil terpenuhi then
            child ← MUTATE(child)
        tambahkan child ke new_population
    population ← new_population
until ada individu yang cukup fit atau waktu habis
return individu terbaik pada population menurut FITNESS-FN
```

Bab 3

Implementasi

3.1 Arsitektur dan Struktur Program

3.2 Representasi State, Fungsi Objektif, dan Constraints

3.3 Representasi Neighbor dan State Awal

3.4 Implementasi Kelas-kelas Pendukung

3.5 Implementasi Algoritma Hill-climbing

3.6 Implementasi Algoritma Simulated Annealing

3.7 Implementasi Algoritma Genetic Algorithm

Bab 4

Hasil Eksperimen dan Analisis

4.1 Eksperimen Algoritma *Hill-climbing*

4.1.1 *Steepest Ascent Hill-climbing*

4.1.2 *Hill-climbing with Sideways Move*

4.1.3 *Random Restart Hill-climbing*

4.1.4 *Stochastic Hill-climbing*

4.2 Eksperimen Algoritma *Simulated Annealing*

4.3 Eksperimen Algoritma *Genetic Algorithm*

4.4 Analisis Hasil Eksperimen

Bab 5

Kesimpulan dan Saran

5.1 Kesimpulan

5.2 Saran

Referensi

Institut Teknologi Bandung. (2026). *Beyond classical search* [Slide kuliah IF3170 Inteligensi Artifisial, Semester I TA 2026/2027]. Program Studi Teknik Informatika, Institut Teknologi Bandung.

Lampiran

Pembagian Tugas

Tabel 1: Pembagian tugas anggota kelompok

No	Nama Anggota	Tugas
1	Nathan E C Marpaung	• AAAA
2	Kurt Mikhael Purba	• AAAAA
3	Aurelia Jennifer Gunawan	• AAAAA

Form Pengunaan AI